Data Collection Kit

Collects architecture and context data from a single repository. Uses **Arcan** for dependency graph analysis and scans context files (README, build configs, dependency manifests, Dockerfiles). Output: one JSONL record per repository.

TL;DR — one command (Docker only)

```
REPO=/full/path/to/myrepo

mkdir -p output

docker run --rm --platform linux/amd64 --network none \
  -v "$(dirname $REPO):/data/repos:ro" \
  -v "$(pwd)/output:/data/arcan_out" \
  -v "$(pwd)/output:/data/output" \
  -e REPO_NAME=$(basename $REPO) \
  -e LANGUAGE=JAVA \
  cezarsas/reverse-se-collector:latest
```

Available languages: `JAVA` · `PYTHON` · `CPP` · `C` · `CSHARP`

Results land in `./output/`:

```
output/
  myrepo.jsonl        ← dataset record
  arcanOutput/
    myrepo/           ← raw Arcan CSVs + graphml
```

To exclude context file categories (e.g. don't share your README or Dockerfiles):

```
docker run --rm --platform linux/amd64 --network none \
  -v "$(dirname $REPO):/data/repos:ro" \
  -v "$(pwd)/output:/data/arcan_out" \
  -v "$(pwd)/output:/data/output" \
  -e REPO_NAME=$(basename $REPO) \
  -e LANGUAGE=JAVA \
  -e EXCLUDE_CATEGORIES=readme,docker \
  cezarsas/reverse-se-collector:latest
```

Available categories: `readme` · `docker` · `build` · `dependencies`

> `--network none` blocks all outbound connections — the container can only read your source files and write to `./output`. Nothing leaves your machine.
>
> Want to try it first? See Try it with an open-source repo below. Want to verify what runs or build your own image? See Build your

own image below.

Note: the Arcan analysis engine is closed source.

Try it with an open-source repo

Run the collector on antlr4 an open-source Java project:

```
git clone --depth 1 https://github.com/antlr/antlr4.git

mkdir -p output

docker run --rm --platform linux/amd64 --network none \
  -v "$(pwd)/antlr4:/data/repos/antlr4:ro" \
  -v "$(pwd)/output:/data/arcan_out" \
  -v "$(pwd)/output:/data/output" \
  -e REPO_NAME=antlr4 \
  -e LANGUAGE=JAVA \
  cezarsas/reverse-se-collector:latest
```

Results land in `./output/antlr4.jsonl`.

Languages

`JAVA · PYTHON · CPP · C · CSHARP`

Output format

```
{
  "repo_id": "myrepo",
  "info": { "language": "java", "hash": "<commit-hash>" },
  "package_stats":  { "num_nodes": 42, "num_edges": 87,
↪  "num_packages": 42 },
  "original_stats": { "num_nodes": 310, "num_edges": 1240,
↪  "num_packages": 42 },
  "package_only_graph": { "nodes": [...], "edges": [...] },
  "original_graph":     { "nodes": [...], "edges": [...] },
  "package_files": [{ "package": "com.example.core", "files":
↪  ["Foo.java"] }],
  "context_files": [
    { "path": "pom.xml",    "category": "dependencies",
↪  "content": "..." },
    { "path": "README.md",  "category": "readme",
↪  "content": "..." }
```

```
  ],
  "node_compression_ratio": 7.38
}
```

Troubleshooting

`No .graphml file` — Arcan failed. Check Docker output; increase heap if needed: `-e JAVA_TOOL_OPTIONS="-Xmx8g"`

`platform does not match` — add `--platform linux/amd64` to your `docker run`.

Empty `arcanOutput/` — check that `--repo` points to valid source code for the given language.

Build your own image

`cezarsas/reverse-se-collector:latest` is built from `Dockerfile.collector` and `docker-entrypoint.sh`. `collect_repo.py` is self-contained (stdlib only) — read it to see exactly what is collected.

To build and push your own image:

```
docker build --platform linux/amd64 -f Dockerfile.collector -t
 ↪  <you>/reverse-se-collector:latest .
docker push <you>/reverse-se-collector:latest
```

Arcan base image: github.com/Arcan-Tech/arcan-2-cli-trial